

3. Spring data jpa HandsOn

Hands-on 1: Introduction to HQL and JPQL

Objective is to Understand the difference between HQL (Hibernate Query Language) and JPQL (Java Persistence Query Language) and learn how they are used to query entities instead of database tables.

Database Table

```
employee
-----
em_id
em_name
em_salary
em_permanent
em_dp_id
```

Entity Class

```
@Entity
@Table(name="employee")
public class Employee {
    @Id
    private int id;
    private String name;
    private double salary;
    private boolean permanent;
}
```

SQL Query

```
SELECT *
FROM employee
WHERE em_permanent = 1;
```

Get all employees

SQL :

```
SELECT * FROM employee;
```

HQL :

```
FROM Employee
```

Get employee by ID

SQL :

```
SELECT *
FROM employee
WHERE em_id = 1;
```

HQL :

```
FROM Employee  
WHERE id = 1;
```

Get permanent employees

SQL :

```
SELECT *  
FROM employee  
WHERE em_permanent = 1;
```

HQL :

```
FROM Employee  
WHERE permanent = true;
```

Get employees with salary greater than 50000

SQL :

```
SELECT *  
FROM employee  
WHERE em_salary > 50000;
```

HQL :

```
FROM Employee  
WHERE salary > 50000;
```

Update salary

SQL :

```
UPDATE employee  
SET em_salary = 60000  
WHERE em_id = 1;
```

HQL :

```
UPDATE Employee  
SET salary = 60000  
WHERE id = 1;
```

Delete employee

SQL :

```
DELETE FROM employee  
WHERE em_id = 5;
```

HQL :

```
DELETE FROM Employee  
WHERE id = 5;
```

JPQL Example :

Repository

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {
    @Query("FROM Employee")
    List<Employee> getAllEmployees();
}
```

Service

```
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository repository;
    @Transactional
    public List<Employee> getAllEmployees() {
        return repository.getAllEmployees();
    }
}
```

Test Method

```
private static void testGetEmployees() {
    LOGGER.info("Start");
    List<Employee> employees =
        employeeService.getAllEmployees();
    LOGGER.debug("Employees : {}", employees);
    LOGGER.info("End");
}
```

Main Method

```
public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);
    employeeService =
        context.getBean(EmployeeService.class);
    testGetEmployees();
}
```

Output

```
INFO Start
DEBUG Employees :
[Employee[id=1,name=John],
 Employee[id=2,name=David],
 Employee[id=3,name=Peter]]
INFO End
```

Hands-on 2: Retrieve All Permanent Employees using Spring Data JPA

Objective is to Implement a service to retrieve all permanent employees from the database using Spring Data JPA with a custom JPQL query.

Step 1: Update EmployeeRepository

Create a custom JPQL query using the @Query annotation.

```
package com.cognizant.ormlearn.repository;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Employee;
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
    @Query("FROM Employee e WHERE e.permanent = true")
    List<Employee> getPermanentEmployees();
}
```

Step 2: Update EmployeeService

Create a service method to fetch permanent employees.

```
package com.cognizant.ormlearn.service;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;
    @Transactional
    public List<Employee> getPermanentEmployees() {
        return employeeRepository.getPermanentEmployees();
    }
}
```

Step 3: Test in OrmLearnApplication

```
private static EmployeeService employeeService;
private static void testGetPermanentEmployees() {
    LOGGER.info("Start");
    List<Employee> employees =
        employeeService.getPermanentEmployees();
    LOGGER.debug("Permanent Employees : {}", employees);
    LOGGER.info("End");
}
```

Step 4: Modify main() Method

```

public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(OrmLearnApplication.class, args);
    employeeService = context.getBean(EmployeeService.class);
    testGetPermanentEmployees();
}

```

JPQL Query Used

```
@Query("FROM Employee e WHERE e.permanent = true")
```

Explanation:

- * Employee → Entity class
- * e → Alias
- * permanent → Java field name
- * true → Fetch only permanent employees

Equivalent SQL Query

```

SELECT *
FROM employee
WHERE em_permanent = 1;

```

Generated SQL (Hibernate)

```

select
  *
from
  employee
where
  em_permanent = true;

```

Output

```

INFO Start
DEBUG Permanent Employees :
Employee[id=1, name=John, salary=80000.0]
Employee[id=2, name=David, salary=65000.0]
INFO End

```

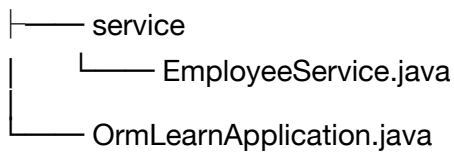
(Output depends on the data available in your database.)

Project Structure

```

src/main/java
|
|—— model
|   |—— Employee.java
|
|—— repository
|   |—— EmployeeRepository.java
|

```



Hands-on 3: Fetch Quiz Attempt Details using HQL

Objective is to Retrieve the details of a quiz attempt using HQL (Hibernate Query Language). The output should include:

- * Username
- * Attempt Date
- * Questions
- * Options for each question
- * Correct Answer
- * Score
- * User Selected Answer

Database Tables

User

user_id

user_name

One

Attempt

attempt_id

attempt_date

user_id

One

AttemptQuestion

aq_id

attempt_id

question_id

Many

Question

question_id

question_text

score

One

Option

option_id

question_id

option_text

```

correct
|
| Many
|
AttemptOption
-----
attempt_option_id
attempt_question_id
option_id
selected

```

Step 1: Create Entity Classes

Create the following entity classes:

```

User.java
Attempt.java
AttemptQuestion.java
Question.java
Option.java
AttemptOption.java

```

Map them using the below things :

```

* @Entity
* @Table
* @OneToMany
* @ManyToOne
* @ManyToMany (if required)

```

Step 2: Create Repository

AttemptRepository.java

```

package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import com.cognizant.ormlearn.model.Attempt;
public interface AttemptRepository extends JpaRepository<Attempt, Integer> {
    @Query("SELECT a FROM Attempt a "
        + "JOIN FETCH a.user u "
        + "JOIN FETCH a.attemptQuestions aq "
        + "JOIN FETCH aq.question q "
        + "JOIN FETCH q.options "
        + "JOIN FETCH aq.attemptOptions ao "
        + "WHERE u.id = :userId "
        + "AND a.id = :attemptId")
    Attempt getAttempt(@Param("userId") int userId,
        @Param("attemptId") int attemptId);
}

```

Step 3: Create Service

AttemptService.java

```

package com.cognizant.ormlearn.service;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Attempt;
import com.cognizant.ormlearn.repository.AttemptRepository;
@Service
public class AttemptService {
    @Autowired
    private AttemptRepository attemptRepository;
    @Transactional
    public Attempt getAttempt(int userId, int attemptId) {
        return attemptRepository.getAttempt(userId, attemptId);
    }
}

```

Step 4: Test Method

OrmLearnApplication.java

```

private static AttemptService attemptService;
private static void testGetAttempt() {
    LOGGER.info("Start");
    Attempt attempt =
        attemptService.getAttempt(1, 1);
    LOGGER.debug("Username : {}",
        attempt.getUser().getName());
    LOGGER.debug("Attempt Date : {}",
        attempt.getAttemptDate());
    for (AttemptQuestion aq :
        attempt.getAttemptQuestions()) {
        LOGGER.debug("Question : {}",
            aq.getQuestion().getText());
        aq.getQuestion().getOptions().forEach(option ->
            LOGGER.debug("{} Score:{} Correct:{}",
                option.getText(),
                option.getScore(),
                option.isCorrect()
            ));
    }
    LOGGER.info("End");
}

```

Step 5: Call the Test

```

public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);
    attemptService =
        context.getBean(AttemptService.class);
    testGetAttempt();
}

```

HQL Query

```

@Query("SELECT a FROM Attempt a "
    + "JOIN FETCH a.user u ")

```



```
+ "JOIN FETCH a.attemptQuestions aq "  
+ "JOIN FETCH aq.question q "  
+ "JOIN FETCH q.options "  
+ "JOIN FETCH aq.attemptOptions ao "  
+ "WHERE u.id = :userId "  
+ "AND a.id = :attemptId")
```

Explanation for HQL Query :

- * JOIN → Joins related entities.
- * FETCH → Loads the related objects immediately.
- * :userId and :attemptId → Named parameters.
- * @Param → Maps method parameters to query parameters.

The document specifically instructs using fetch for one-to-many or many-to-many relationships so that the associated objects are populated.

Output

```
INFO Start  
Username : John  
Attempt Date : 20-06-2026  
Question :  
What is HTML?  
Option 1 : .xhtm  
Score : 0  
Correct : false  
Option 2 : .html  
Score : 1  
Correct : true  
Option 3 : .ht  
Score : 0  
Correct : false  
Question :  
Maximum heading tag?  
Option 1 : h3  
Score : 0  
Option 2 : h6  
Score : 1  
INFO End
```

Hands-on 4: Difference between JPA, Hibernate, and Spring Data JPA

Objective is to Understand the difference between:

- * JPA (Java Persistence API)
- * Hibernate
- * Spring Data JPA

And implement a simple example using Spring Data JPA to compare it with traditional Hibernate code.

1. JPA (Java Persistence API)

JPA is a Java Specification (JSR 338) for Object Relational Mapping (ORM).

- * It defines interfaces and annotations.
- * It does not contain any implementation.
- * It requires an implementation such as Hibernate.

Example annotations:

```
@Entity
@Table(name = "employee")
public class Employee {
    @Id
    private int id;
    private String name;
}
```

2. Hibernate

Hibernate is an ORM Framework.

It is one of the implementations of JPA.

Hibernate provides:

- * SessionFactory
- * Session
- * Transaction
- * HQL
- * Caching
- * Lazy Loading

Example (Traditional Hibernate)

```
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeId = null;
    try {
        tx = session.beginTransaction();
        employeeId = (Integer) session.save(employee);
        tx.commit();
    } catch (Exception e) {
        if (tx != null)
            tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeId;
}
```

3. Spring Data JPA

Spring Data JPA is built on top of JPA.

It reduces boilerplate code.

Instead of writing Session, Transaction, Save, Close manually, Spring handles everything automatically.

Example Repository

```
package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Employee;
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

Employee Service

```
package com.cognizant.ormlearn.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;
    @Transactional
    public void addEmployee(Employee employee) {
        employeeRepository.save(employee);
    }
}
```

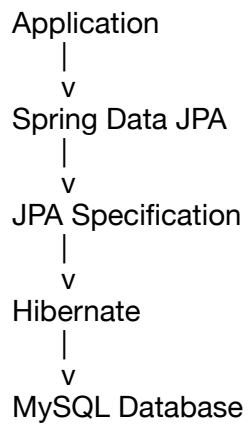
Test in OrmLearnApplication

```
private static EmployeeService employeeService;
private static void testAddEmployee() {
    LOGGER.info("Start");
    Employee employee = new Employee();
    employee.setId(5);
    employee.setName("Rahul");
    employee.setSalary(60000);
    employee.setPermanent(true);
    employeeService.addEmployee(employee);
    LOGGER.info("Employee Added Successfully");
    LOGGER.info("End");
}
```

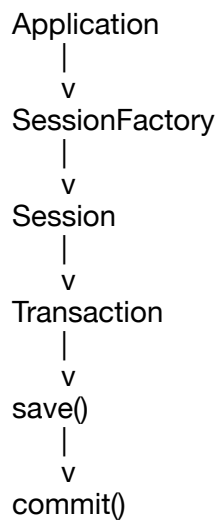
Main Method

```
public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);
    employeeService =
        context.getBean(EmployeeService.class);
    testAddEmployee();
}
```

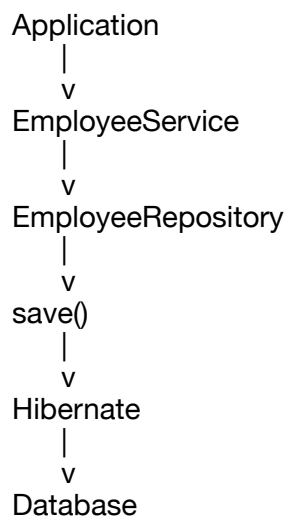
Architecture



Hibernate Flow



Spring Data JPA Flow



Built-in Repository Methods

save()
findAll()

```
findById()
deleteById()
count()
existsById()
```

Output

```
INFO Start
INFO Employee Added Successfully
INFO End
```

The employee record will be inserted into the employee table.

Hands-on 5: Implement Services for Managing Country

Objective is to Implement the following CRUD operations using Spring Data JPA:

- * Find Country by Code
- * Add Country
- * Update Country
- * Delete Country
- * Find Countries by Partial Name

Step 1: Configure Hibernate

application.properties

```
# Hibernate DDL Auto
spring.jpa.hibernate.ddl-auto=validate
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root
# Hibernate Dialect
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
# Show SQL
spring.jpa.show-sql=true
```

ddl-auto Options

Value	Description
create	Drops existing tables and creates new ones
update	Updates database schema if required
validate	Validates table structure only
create-drop	Creates tables and drops them when the application stops

Step 2: Country Entity

Country.java

```
package com.cognizant.ormlearn.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
@Entity
```

```

@Table(name = "country")
public class Country {
    @Id
    @Column(name = "co_code")
    private String code;
    @Column(name = "co_name")
    private String name;
    public Country() {
    }
    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public String getCode() {
        return code;
    }
    public void setCode(String code) {
        this.code = code;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

Step 3: Repository

CountryRepository.java

```

package com.cognizant.ormlearn.repository;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContainingIgnoreCase(String name);
}

```

Step 4: Country Service

CountryService.java

```

package com.cognizant.ormlearn.service;
import java.util.List;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
@Service

```

```

public class CountryService {
    @Autowired
    private CountryRepository countryRepository;
    @Transactional(readOnly = true)
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
    @Transactional(readOnly = true)
    public Country findCountryByCode(String code) {
        Optional<Country> result = countryRepository.findById(code);
        return result.orElse(null);
    }
    @Transactional
    public void addCountry(Country country) {
        countryRepository.save(country);
    }
    @Transactional
    public void updateCountry(String code, String name) {
        Optional<Country> result = countryRepository.findById(code);
        if (result.isPresent()) {
            Country country = result.get();
            country.setName(name);
            countryRepository.save(country);
        }
    }
    @Transactional
    public void deleteCountry(String code) {
        countryRepository.deleteById(code);
    }
    @Transactional(readOnly = true)
    public List<Country> searchCountry(String name) {
        return countryRepository.findByNameContainingIgnoreCase(name);
    }
}

```

Step 5: Testing in OrmLearnApplication

```

private static CountryService countryService;

```

Test 1: Get All Countries

```

private static void testGetAllCountries() {
    LOGGER.info("Start");
    List<Country> countries = countryService.getAllCountries();
    countries.forEach(System.out::println);
    LOGGER.info("End");
}

```

Test 2: Find Country by Code

```

private static void testFindCountry() {
    LOGGER.info("Start");
    Country country = countryService.findCountryByCode("IN");
    System.out.println(country);
    LOGGER.info("End");
}

```

Test 3: Add Country

```
private static void testAddCountry() {
    LOGGER.info("Start");
    Country country = new Country("NP", "Nepal");
    countryService.addCountry(country);
    System.out.println("Country Added");
    LOGGER.info("End");
}
```

Test 4: Update Country

```
private static void testUpdateCountry() {
    LOGGER.info("Start");
    countryService.updateCountry("NP", "Federal Democratic Republic of Nepal");
    System.out.println(countryService.findCountryByCode("NP"));
    LOGGER.info("End");
}
```

Test 5: Delete Country

```
private static void testDeleteCountry() {
    LOGGER.info("Start");
    countryService.deleteCountry("NP");
    System.out.println("Country Deleted");
    LOGGER.info("End");
}
```

Test 6: Search Country

```
private static void testSearchCountry() {
    LOGGER.info("Start");
    List<Country> countries =
        countryService.searchCountry("Ind");
    countries.forEach(System.out::println);
    LOGGER.info("End");
}
```

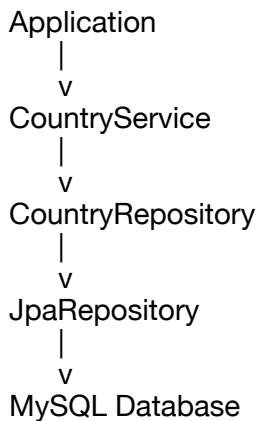
Main Method

```
public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);
    countryService =
        context.getBean(CountryService.class);
    testGetAllCountries();
    testFindCountry();
    testAddCountry();
    testUpdateCountry();
    testSearchCountry();
    testDeleteCountry();
}
```


Repository Methods Used are :

```
findAll()
findById()
save()
deleteById()
findByNameContainingIgnoreCase()
```

Flow Diagram



Output

```
INFO Start
Country [code=IN, name=India]
Country Added
Country [code=NP, name=Federal Democratic Republic of Nepal]
Country [code=IN, name=India]
Country Deleted
INFO End
```

Hands-on 6: Find a Country Based on Country Code

Objective is to Implement a service method to find a country using its country code with exception handling.

Step 1: Create Exception Class

Create package:

```
com.cognizant.ormlearn.service.exception
```

```
CountryNotFoundException.java
```

```
package com.cognizant.ormlearn.service.exception;
public class CountryNotFoundException extends Exception {
    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

Step 2: Update CountryService

CountryService.java

```
package com.cognizant.ormlearn.service;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;
@Service
public class CountryService {
    @Autowired
    private CountryRepository countryRepository;
    @Transactional
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {
        Optional<Country> result = countryRepository.findById(countryCode);
        if (!result.isPresent()) {
            throw new CountryNotFoundException(
                "Country with code " + countryCode + " not found");
        }
        Country country = result.get();
        return country;
    }
}
```

Step 3: Test Method in OrmLearnApplication

```
private static void testFindCountryByCode() {
    LOGGER.info("Start");
    try {
        Country country =
            countryService.findCountryByCode("IN");
        LOGGER.debug("Country : {}", country);
    } catch (CountryNotFoundException e) {
        LOGGER.error(e.getMessage());
    }
    LOGGER.info("End");
}
```

Step 4: Call Test Method

```
public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(OrmLearnApplication.class, args);
    countryService = context.getBean(CountryService.class);
    testFindCountryByCode();
}
```

Output (Country Exists)

```
INFO Start
DEBUG Country : Country [code=IN, name=India]
INFO End
```

Output (Country Does Not Exist)

If you search for:

```
countryService.findCountryByCode("XX");
```

Output:

```
INFO Start  
ERROR Country with code XX not found  
INFO End
```

Repository Method Used

```
Optional<Country> result =  
    countryRepository.findById(countryCode);
```

Flow Diagram

```
User  
|  
v  
findCountryByCode("IN")  
|  
v  
CountryRepository.findById()  
|  
v  
Optional<Country>  
|  
+--> Present -----> Return Country  
|  
+--> Empty -----> Throw CountryNotFoundException
```